RxInsight: Interview Questions

Updated 29 October 2026. Answers match repaired implementation `da15d2c`. Understand the reasoning rather than memorising a script. Results are synthetic-data software measurements.

1. Project and business understanding

What does the project do?

It builds a PostgreSQL warehouse from generated HCP, prescription and call extracts. Python validates records, preserves HCP history, resolves temporal keys and reconciles facts/rejects. SQL reports answer prescriber, therapeutic-market share, territory and call-association questions.

What is the strongest engineering result?

The full load accounts for 2,230,247 prescription inputs as 2,229,747 facts and 500 genuine malformed rejects. Independent source/warehouse aggregates had zero mismatches across four measures. That is stronger evidence than successful execution alone.

Is this real pharmaceutical data?

No. A seeded generator creates structured synthetic data, a known call-related signal and deliberate malformed rows. Do not describe the output as a real commercial finding or a patient study.

Why a warehouse instead of one flat table?

Shared dimensions give consistent business definitions across reports. The two facts retain different grains. Flattening everything would either duplicate measures or bury history and grain choices in individual reports.

2. Modelling and history

What is grain, and why declare it first?

It states exactly what one row means. Prescription facts represent HCP version/product/day; call facts represent recorded calls. A join can otherwise multiply rows and inflate totals while looking plausible.

Natural key versus surrogate key?

`hcp_id` follows the person across time. `hcp_key` identifies one warehouse version. Historical territory reporting needs the version; longitudinal person-level exposure needs the stable identity.

> Analogy: a person keeps their ID after moving house, while an old delivery receipt keeps its original address. Stable identity and historical address-version identity answer different questions.

Why SCD Type 2?

Overwriting territory would move old prescriptions into the new geography. Type 2 closes an old interval and adds a current version, allowing each event date to resolve to its historical attributes.

What enforces valid history?

A partial unique index allows at most one current version per HCP. A CHECK rejects inverted intervals. Tests check overlaps and fact dates against history. The transform rejects conflicting same-day/backdated tracked changes. No single check proves every invariant.

Does the partial index guarantee exactly one current version?

It guarantees at most one, not at least one. A separate loaded-data assertion checks the current-row invariant. Distinguish database enforcement from tested assumptions.

Is the complete loader incremental?

No. The normal command rebuilds staging and warehouse schemas and folds supplied history into a fresh dimension. A pure SCD function accepting existing state does not provide ingestion watermarks, checkpoints or restartable incremental publication.

Can a late March fact find the March version?

Yes, resolution is by event date. That does not imply there is an implemented backfill workflow for rewriting dimension history or a late-data ingestion process.

Why use Python dates for the sentinel?

The year-9999 endpoint overflows a pandas 2.x nanosecond datetime column. Python dates preserve it on both tested dependency configurations. Shortening the history horizon was not required.

3. Validation and loading

Why is staging TEXT?

So raw malformed values can be landed and rejected with explanations before typed fact loading. TEXT staging does not make later direct casts safe; calendar-boundary casts must also be guarded.

Which bad values are covered?

Missing, fractional, non-finite and overflowing integer values; unknown keys; malformed dates; temporal gaps; and ambiguous matching history. Missing requested columns are schema errors. Destination-specific bounds are applied to known integer fields.

What caused temporal row loss?

A known HCP with no valid version was removed by the temporal candidate filter. It reached neither facts nor rejects. The repair joins valid matches back to every input position, preserving unresolved rows for quarantine.

Why does iloc matter?

Earlier rejects leave gaps in index labels. The resolver tracks positions, so label-based loc can select the wrong identities. Counts can still balance; the regression must check which records were rejected.

What is the reconciliation identity?

Staged equals loaded plus rejected. Here that is 2,230,247 = 2,229,747 + 500. Also verify the reject reasons and identities, because swapping clean and malformed rows preserves arithmetic.

Why COPY?

It streams buffered records with less command overhead than separate INSERTs. It is appropriate for validated batches. It does not validate all business meaning or eliminate the cost of serialising data.

Why remove and restore fact foreign keys?

The load avoids repeated per-row FK checks, then restores all eight constraints with validation before committing the facts. A violation fails restoration. Leaving constraints disabled permanently would be a different and unsafe design.

Is the whole refresh atomic?

No. Earlier stages commit. The fact load, FK revalidation and reconciliation are protected before the final fact commit. A production version might build a new warehouse version and publish it atomically; that is future work.

Why server-side cursors as well as chunksize?

Chunksize gives pandas batches; a server-side cursor prevents the driver from first buffering the entire query result. This bounds fact reads, while generation and reject accumulation have separate memory costs.

4. SQL and business definitions

Why partition brand share by therapeutic area?

Cardiology share should use the tracked cardiology market. With 60 own-brand and 40 competitor TRx, share is 60%. Adding 100 unrelated TRx must not turn that answer into 30%.

What do LAG, RANK and NTILE do here?

LAG supplies preceding reported share; RANK gives tied volumes the same rank; NTILE makes approximately equal-sized groups within territory. Correct partitions and grains remain essential.

Why can stored and calculated deciles disagree?

The stored value increases toward high volume; descending NTILE starts at 1 for high volume. They also arise from different definitions. Align direction and logic before comparing values.

Where do targets come from?

The first six observed own-brand monthly totals, averaged and increased by 8%. It is a demonstration quota, not a real annual-plan feed. The first six months are labelled baseline periods rather than independent performance evidence.

What was the quota-rounding defect?

An intermediate repeating decimal could land below an exact half boundary. Multiplying before dividing gives `8675 * 1.08 / 6 = 1561.50`, rounded to 1562. A SQL regression now protects that case.

Why does call exposure use hcp_id?

A call before a territory transfer can precede the same person's later prescription. The surrogate changes while the person ID remains. Historical territory reporting still uses the version, because that report asks a different question.

What is a called person-month?

A prescribing person-month with at least one prescribing day preceded by a call 1 to 14 days earlier. Same-day calls are excluded. A person is counted once across versions; its segment is the decile valid on the last prescribing day.

What does the measured call result establish?

The repaired report has 39,225 called person-months averaging 39.32 TRx and 7,959 uncalled months averaging 23.70. A separate raw-CSV as-of calculation matched both. This establishes implementation consistency, not causal promotional benefit.

Why does decile stratification not prove causality?

It does not remove all differences between cohorts. Exposure is not randomised; non-prescribing months are excluded; the generator plants a signal. Do not call the association a causal estimate or a guaranteed causal upper bound.

5. Performance and evidence

Which index measurably helped?

The selective prescriber lookup improved from 131.640 ms to 2.250 ms, about 58.5x in this environment, using a Bitmap Heap Scan. The full aggregate stayed a Parallel Seq Scan without material improvement. Quote the workload and conditions with the number.

When is a sequential scan correct?

When most rows are needed, sequential reading and parallel aggregation can beat index traversal plus heap access. An index earns its keep through selectivity, useful coverage or ordering, not table size alone.

How was cache bias reduced?

Discard a warm-up and take the median of three runs per state. Compare the same query and data without unrelated heavy work. Absolute times still depend on machine state. The benchmark aggregate is simplified, not the whole window report.

What was tested?

There are 36 pure unit cases, 21 loaded-data cases and 10 SQL acceptance cases: 67 per dependency configuration. CI requires the DB. The audit additionally compared source/warehouse aggregates and report arithmetic and recomputed call cohorts through a different algorithm.

How do you prevent skipped DB checks from looking like a full pass?

Make and CI use `--require-db`, making an unavailable or empty warehouse fail. The pure unit subset can run without a database, but it is not presented as complete verification.

What would you change at much larger scale?

Measure the bottleneck first. Consider incremental ingestion, safe publication, late-data handling and partition pruning for bounded date predicates. Partitioning does not automatically speed an all-history aggregate, and sharding adds complexity rather than being an automatic next step.

What should you never claim?

Real patient data, causal call uplift, a portable 58.5x speedup, removal of scans from every report, production incremental loading, or proof of every deployment path from a unit-test count.

Practice

Trace a transfer with two dates; trace a malformed row into a reject file; calculate 60% market share and the 1562 quota; explain stable-person versus historical-version identity; then compare selective and full-scan execution plans.

Sources: Reviewed implementation | Verification record | Architecture guide